

Tr_12 – Diagrama de Clases UML

Documento de Referencia: **Diagrama de Clases UML**

Proyecto: Aplicación “**No Te Cortes**”

Sector: **Servicios de Estética e Imagen Personal (CNAE 9602)**

1. Justificación Técnica

El diagrama representa la estructura estática de la aplicación utilizando la notación **UML 2.5**. Se ha diseñado una arquitectura basada en **POJOs** (Plain Old Java Objects) para facilitar la persistencia en bases de datos NoSQL como Firebase Firestore.

Se han aplicado los siguientes principios de Programación Orientada a Objetos (POO):

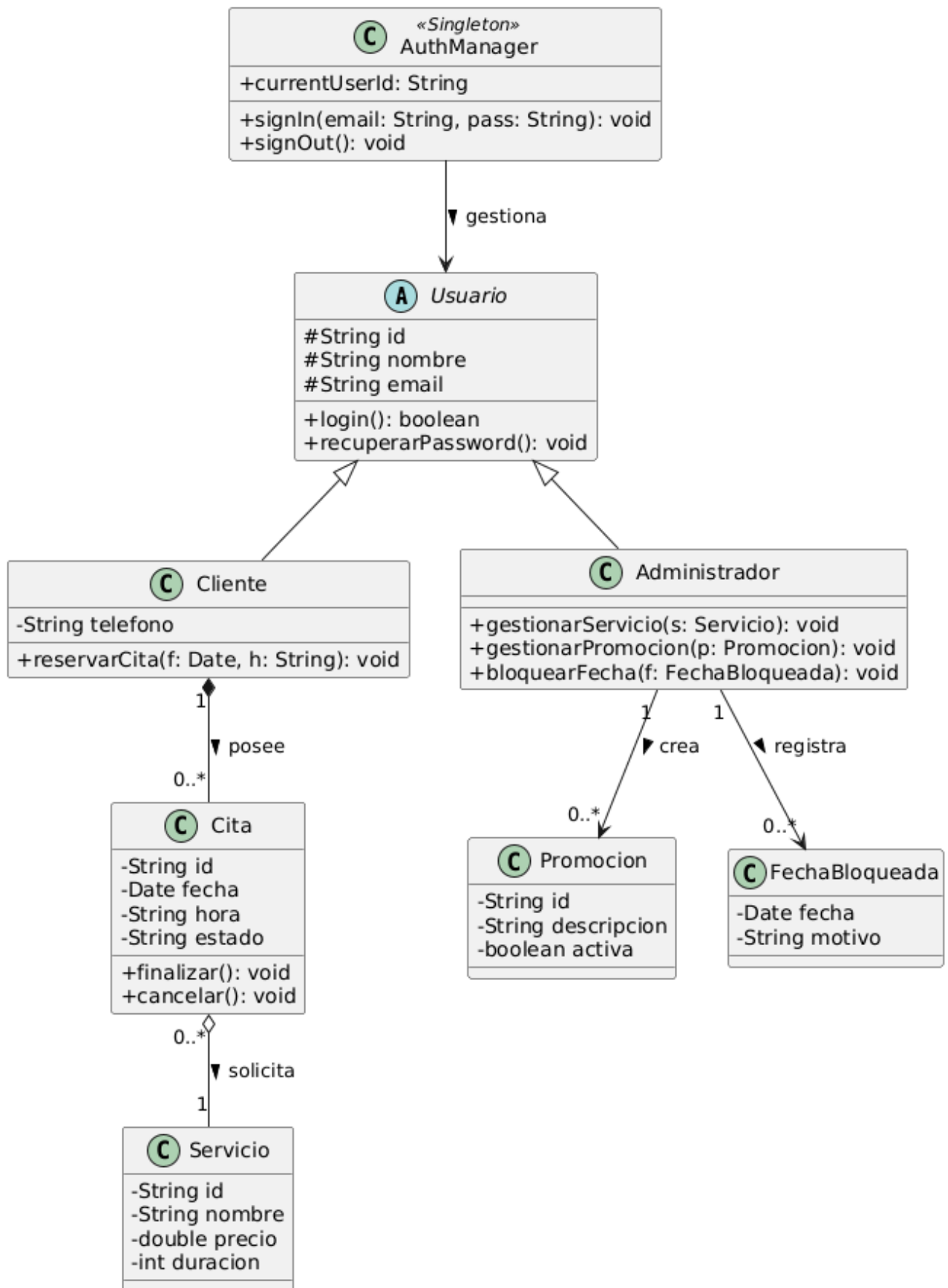
- **Encapsulamiento:** Uso de visibilidad privada (-) para proteger los datos y pública (+) para los métodos de interfaz.
- **Abstracción:** La clase Usuario se define como abstracta, ya que no se instancian usuarios genéricos, sino perfiles específicos.
- **Especialización:** Uso de herencia para que Cliente y Administrador compartan la lógica base de sesión.

2. Definición de Clases y Relaciones

Para este diseño, aplicamos una arquitectura basada en **Modelos (POJOs)** y **Controladores**.

- **Herencia:** Centralizamos en Usuario la lógica de sesión y datos básicos. Cliente, Empleado y Administrador especializan las acciones.
- **Agregación (Cita - Servicio):** Una Cita necesita un Servicio. Es agregación porque el servicio existe independientemente de si hay citas o no.
- **Composición (Usuario - Cita):** En el contexto de nuestra App, una Cita no tiene sentido sin un Usuario que la posea.

3. Diagrama de Clases



4. Descripción de Componentes

A. Atributos y Métodos

- **Clase Cita:** Incluye atributos técnicos como id, fecha, hora y estado. Sus métodos (`finalizar()`, `cancelar()`) no devuelven valor (`void`) y operan sobre el estado del objeto.
- **Clase Administrador:** Posee métodos de gestión específicos con parámetros definidos: `gestionarServicio(s: Servicio)` y `gestionarPromocion(p: Promocion)`.
- **Clase AuthManager:** Se implementa como **Singleton** para garantizar una única instancia de gestión de sesión en toda la App.

Nota de diseño: La entidad *PASSWORD_RESET* definida previamente en el modelo conceptual se ha integrado en este diagrama mediante el método `recuperarPassword()` dentro de la clase `Usuario`. Esta decisión busca simplificar la lógica de negocio y aprovechar la integración nativa de Firebase Auth para la gestión de credenciales.

B. Relaciones y Cardinalidad

- **Cliente - Cita (1 : 0..*):** Un cliente puede no tener citas o tener muchas. Cada cita pertenece obligatoriamente a **1** cliente.
- **Cita - Servicio (0.. : 1):*** Una cita solicita exactamente **1** servicio. Un servicio específico puede estar presente en muchas citas diferentes.
- **Administrador - Promoción/Fecha (1 : 0..*):** Un administrador registra y gestiona múltiples promociones y bloqueos de calendario.